Real-Time Vehicle Tracking & Counting

An AI-Powered Computer Vision Project

Powered by: YOLO11 & OpenCV

Date: August 2026

1. Introduction

Have you ever wondered how smart cities monitor traffic, or how navigation apps know when a road is congested? It all starts with cameras and intelligent software that can actually "see" and count vehicles. That was the inspiration behind this project.

Instead of manually sitting and tallying cars as they drive by, I built a system that uses a live video feed to automatically detect, track, and count different types of vehicles in real time. The goal was to create something that doesn't just draw boxes around objects, but actually understands what it's looking at—differentiating between a bicycle, a car, and a large truck.

2. How the Brain of the Project Works

To make the program smart, I used an Artificial Intelligence model called **YOLO11** (which stands for "You Only Look Once"). Created by a company called Ultralytics, YOLO is incredibly fast, making it perfect for processing video where every millisecond counts.

The model I chose (`yolo11l.pt`) is a "pre-trained" model. This means I didn't have to spend thousands of hours teaching a computer what a car looks like. The AI was previously trained on a massive library of images called the COCO dataset. Out of the box, it knows how to recognize over 80 everyday items. For this project, I filtered its focus so it only pays attention to vehicles: bicycles, cars, motorcycles, buses, and trucks.

3. The Logic: Setting the "Tripwire"

Detecting a car is one thing, but counting it accurately as it moves across the screen is a completely different challenge. If the AI looks at a video running at 30 frames per second, it sees the same car 30 times in one second. If we just counted detections, one car would be counted dozens of times!

> **The Solution:** I gave the AI a memory. By enabling YOLO's built-in tracking feature, the AI assigns a unique ID to every new vehicle it spots and follows that ID across the screen.

Next, I drew a virtual "tripwire" (a red line) across the video feed. The program constantly calculates the exact center point (the centroid) of every vehicle's bounding box. The rule I programmed is simple: **Only count a vehicle when its center point crosses over that red line.**

To be absolutely sure nothing is double-counted, once a vehicle crosses the line, its unique ID is stored in a special list called a "set". The program checks this list every frame, ensuring that no ID is ever counted twice.

4. Technologies Used

- **Python:** The main programming language that ties everything together.
- **Ultralytics (YOLO11):** The AI engine responsible for detecting and tracking the vehicles.
- **OpenCV (cv2):** The computer vision library used to read the video file, draw the red line, overlay text, and display the live feed.
- **Collections (defaultdict):** A smart Python dictionary used to safely store the tallies for each vehicle type without crashing the program when a new vehicle class appears.

5. Challenges Overcome

One of the subtle challenges was handling the vehicle counts dynamically. In standard programming, if the system tries to add a tally to "bus" before any bus has been detected, the program will crash. By using Python's `defaultdict`, I made the program robust enough to dynamically add new vehicle categories on the fly, starting their count at zero safely before adding one.

6. Conclusion and Future Scope

This project successfully demonstrates how accessible and powerful modern computer vision has become. With less than 100 lines of code, the system acts as a highly accurate, tireless traffic monitor.

In the future, this project could easily be expanded. By drawing two lines instead of one, the system could calculate the time it takes for a vehicle to travel between them, thereby measuring the vehicle's speed. It could also be adapted to detect traffic in multiple lanes or recognize license plates for automated toll booths.